

PROJET HELIOS

MERCREDI CYBER — 07 OCTOBRE 2026

ACOUSTIC FINGERPRINTING

De l'onde sonore à l'identification musicale :
modélisation mathématique, construction des empreintes, indexation et recherche par alignement temporel

PAPIER DE RECHERCHE TECHNIQUE

Objet. Étudier en détail le principe algorithmique qui permet à un système de type Shazam d'identifier un enregistrement à partir d'un court extrait capturé par microphone.

Thèse centrale. Le système ne compare pas naïvement des formes d'onde complètes. Il transforme le son en une représentation temps-fréquence parcimonieuse, construit des empreintes compactes, les indexe et recherche une cohérence temporelle entre les occurrences.

Élément	Description
Problème	Identifier un extrait audio bruité parmi une très grande bibliothèque.
Chaîne	Échantillonnage → STFT → spectrogramme → pics → constellation → hash → index → offsets → score.
Référence scientifique	Avery Wang, An Industrial Strength Audio Search Algorithm, ISMIR 2003.
Technologie industrielle	Shazam / Apple ShazamKit et autres solutions ACR.
Important	Les détails internes exacts de Shazam sont propriétaires ; les paramètres numériques présentés ici sont pédagogiques.

Mots-clés : acoustic fingerprinting, audio fingerprint, Shazam, FFT, STFT, spectrogramme, peak picking, constellation map, hashing, index inversé, offset temporel, ACR.

Résumé

L'identification automatique d'un morceau de musique à partir d'un téléphone est un problème de traitement numérique du signal et de recherche à grande échelle. Une comparaison directe des échantillons audio serait à la fois coûteuse et fragile : le microphone introduit du bruit, la pièce ajoute de la réverbération, la compression modifie le signal et le niveau sonore peut varier. L'acoustic fingerprinting répond à ce problème en recherchant une représentation stable et compacte du contenu.

Le présent papier décrit une reconstruction pédagogique détaillée d'un algorithme de type Shazam. Le signal est échantillonné puis analysé par transformée de Fourier à court terme (STFT). Le spectrogramme obtenu est parcouru pour détecter des maxima locaux. Ces maxima forment une carte de constellation. Des relations entre points sont ensuite encodées sous forme de clés de hachage. Une base de fingerprints est organisée comme un index inversé : chaque clé renvoie vers les morceaux et positions où elle apparaît. Lors d'une requête, les mêmes fingerprints sont calculés puis recherchés. Les décalages temporels Δt sont accumulés ; un pic dominant de l'histogramme indique un alignement cohérent et fournit la preuve principale de l'identification.

Apple décrit publiquement ShazamKit comme utilisant une signature acoustique compacte qui capture la distribution temps-fréquence de l'énergie audio. La documentation indique également que des signatures de requête sont comparées à des signatures de référence et que la recherche peut fonctionner sur du son capturé avec du bruit. Des catalogues personnalisés peuvent être utilisés pour des contenus audio ou vidéo, notamment pour des expériences synchronisées.

Attention : ce document explique les principes publics et une reconstruction algorithmique reproductible. Il ne prétend pas révéler le code source ou les paramètres propriétaires actuels de Shazam.

1. Introduction

Un son numérique $x[n]$ peut être vu comme une longue suite de nombres. Cette représentation est excellente pour la lecture, mais peu adaptée à une recherche massive. Le fingerprinting cherche donc à répondre à une question différente : quelles petites structures du signal permettent de reconnaître le même contenu après une capture imparfaite ?

La réponse étudiée ici est de travailler dans le domaine temps-fréquence. Une composante musicale n'est pas seulement une fréquence ; elle apparaît à un instant, avec une certaine énergie, puis évolue. Le spectrogramme permet de représenter cette évolution. Le système sélectionne ensuite seulement les points les plus informatifs et encode leurs relations.

2. Fondements du signal numérique

2.1. Échantillonnage

Un signal analogique $x(t)$ devient une suite $x[n]$ lorsqu'il est échantillonné à une fréquence f_s . Le pas temporel entre deux échantillons vaut :

$$\Delta t_s = 1 / f_s$$

Avec $f_s = 8\,000$ Hz, $\Delta t_s = 0,125$ ms. Avec 44,1 kHz, le pas est environ 0,0227 ms. Une fréquence élevée fournit plus d'échantillons mais augmente aussi le coût de traitement.

2.2. Nyquist-Shannon

Pour représenter une bande limitée à $f_{\max}$, la fréquence d'échantillonnage doit respecter la condition de Nyquist, approximativement $f_s \geq 2f_{\max}$. Le système peut ensuite rééchantillonner le signal selon la bande utile à son algorithme.

2.3. Fenêtrage

La transformée de Fourier est généralement calculée sur des blocs de N échantillons. Une fenêtre $w[n]$ est appliquée pour réduire les discontinuités artificielles aux bords : $x_w[n] = x[n]w[n]$. Les fenêtres successives se recouvrent souvent. H désigne alors le hop size, c'est-à-dire le déplacement entre deux fenêtres.

Exemple pédagogique : $f_s = 8$ kHz, $N = 1024$, $H = 512$. La fenêtre dure 128 ms et le déplacement est 64 ms.

3. FFT et STFT : passer du temps à la fréquence

3.1. DFT et FFT

La Transformée de Fourier Discrète (DFT) décompose un bloc temporel en composantes fréquentielles :

$$X[k] = \sum_{n=0}^{N-1} x[n]e^{-j2\pi kn/N}$$

La FFT (Fast Fourier Transform) est une famille d'algorithmes permettant de calculer efficacement cette DFT, typiquement en $O(N \log N)$ pour les tailles favorables.

3.2. Pourquoi la FFT seule est insuffisante

Une FFT globale indique quelles fréquences sont présentes mais perd leur localisation temporelle. Pour identifier un extrait, cette information est insuffisante. La STFT répète donc la FFT sur des fenêtres successives.

3.3. Formule de la STFT

$$X(m,k) = \sum_{n=0}^{N-1} x[n+mH]w[n]e^{-j2\pi kn/N}$$

m représente la trame temporelle, k le bin fréquentiel, N la taille de FFT et H le hop size. La résolution fréquentielle approximative vaut $\Delta f = f_s/N$. Pour 8 kHz et 1024 points, $\Delta f \approx 7,8125$ Hz.

3.4. Compromis temps/fréquence

Une grande fenêtre améliore la résolution fréquentielle mais réduit la précision temporelle. Une petite fenêtre fait l'inverse. Le choix est donc un compromis fondamental de la chaîne de fingerprinting.

4. Spectrogramme et représentation temps-fréquence

4.1. Construction

À partir de la STFT, on calcule une grandeur d'énergie ou d'amplitude, par exemple $S(m,k)=|X(m,k)|^2$. La matrice S est le spectrogramme.

4.2. Lecture du spectrogramme

L'axe horizontal représente le temps, l'axe vertical la fréquence et l'intensité représente l'énergie. Une note ou un événement musical apparaît donc comme une structure localisée dans ce plan.

4.3. Pourquoi ne pas conserver toute la matrice ?

Une chanson de plusieurs minutes produit un grand nombre de cellules. Le fingerprinting cherche au contraire une représentation parcimonieuse. L'objectif est de garder des événements rares et robustes plutôt que toutes les valeurs.

Le passage spectrogramme → points caractéristiques est donc le premier grand gain de compression conceptuelle.

5. Peak Picking : sélection des maxima locaux

5.1. Définition

Un pic est une cellule dont l'énergie est supérieure à celle d'un voisinage et à un seuil. Une formulation simplifiée est : $S(m,k) \geq S(i,j)$ pour tout voisin (i,j) et $S(m,k) \geq \tau$.

5.2. Paramètres importants

Le seuil τ contrôle la sensibilité. Le voisinage contrôle la notion de maximum local. Une distance minimale entre pics évite une densité excessive. Des seuils adaptatifs peuvent être utilisés lorsque le niveau énergétique varie beaucoup.

5.3. Résultat

On obtient $P=\{(t_i, f_i, a_i)\}$. Cette liste est beaucoup plus petite que la matrice complète du spectrogramme. La représentation est donc parcimonieuse.

5.4. Pourquoi les pics sont utiles

Les pics correspondent à des structures énergétiques dominantes. Ils sont généralement plus stables sous certaines variations de volume que les amplitudes absolues. Ils constituent aussi des événements faciles à indexer.

6. Carte de constellation

6.1. Principe

Si les pics sont représentés uniquement par (t, f) , on obtient une constellation. Deux enregistrements du même morceau doivent produire des configurations proches, même si certaines amplitudes ont changé.

6.2. Pourquoi une fréquence seule ne suffit pas

Une fréquence comme 440 Hz apparaît dans énormément de contenus. En revanche, la combinaison de plusieurs fréquences et de leurs distances temporelles est beaucoup plus discriminante.

6.3. Relations entre points

Pour un point d'ancrage $p1=(t1, f1)$ et une cible $p2=(t2, f2)$, on calcule $\Delta t=t2-t1$. Une relation peut donc être décrite par $(f1, f2, \Delta t)$. Le temps absolu de l'ancre reste stocké séparément afin de retrouver la position dans le morceau.

Exemple : $(0, 440)$, $(0,13, 554)$ donne $\Delta t=0,13$ s ; une autre paire peut avoir une autre relation. La combinaison devient une signature locale.

7. Hashing des relations

7.1. Clé de fingerprint

Une modélisation pédagogique est : $H = \text{hash}(f1, f2, \Delta t)$. Les fréquences et le délai sont quantifiés afin de limiter le nombre de valeurs possibles.

7.2. Pourquoi le hash accélère la recherche

Une table de hachage permet de passer directement d'une clé à ses occurrences. Au lieu de comparer une requête avec tous les morceaux, le système demande : « où cette structure a-t-elle déjà été observée ? »

```
anchor = (t1, f1)
target = (t2, f2)
dt = t2 - t1
key = HASH(QUANTIZE(f1),
           QUANTIZE(f2),
           QUANTIZE(dt))
index[key].append((track_id, t1))
```

7.3. Fenêtre de cibles

Chaque ancre n'est pas reliée à tous les pics de la chanson. Une fenêtre $[\Delta_{\min}, \Delta_{\max}]$ limite les cibles. Cela contrôle la densité de l'empreinte et donc la mémoire nécessaire.

7.4. Limite

Cette formule est une reconstruction pédagogique. Elle illustre le principe de relations de constellation et de hachage ; elle ne doit pas être présentée comme la formule propriétaire exacte utilisée actuellement par Shazam.

8. Index inversé

8.1. Structure

L'index associe une clé à toutes ses occurrences : $H \rightarrow [(track_id, t)]$. Cette structure est dite inversée parce qu'on part d'une caractéristique pour retrouver les documents/contenus qui la contiennent.

Hash	Morceau	Position
8F2A	Track A	12,40 s
8F2A	Track C	41,80 s
4C91	Track A	12,53 s
A713	Track A	12,68 s

8.2. Construction hors ligne

Les morceaux de référence sont analysés une fois, puis leurs fingerprints sont ajoutés à l'index. Lorsqu'un nouveau morceau entre dans la bibliothèque, seule sa phase d'ingestion doit être recalculée.

```
for track in library:
    x = load(track)
    X = STFT(x)
    S = abs(X)**2
    peaks = peak_picking(S)

    for anchor in peaks:
        for target in targets(anchor, peaks):
            dt = target.t - anchor.t
            H = hash(q(anchor.f), q(target.f), q(dt))
            index[H].append((track.id, anchor.t))
```

9. Algorithme de reconnaissance

9.1. Pipeline de requête

Le microphone fournit un extrait. Le système applique la même chaîne de traitement : normalisation/rééchantillonnage éventuel, STFT, spectrogramme, peak picking, construction de relations et fingerprints.

```
query_fingerprints = fingerprint(query_audio)

for (Hq, tq) in query_fingerprints:
    for (track_id, td) in index.get(Hq, []):
        delta = td - tq
        votes[track_id].append(delta)
```

9.2. Le rôle fondamental de Δt

Pour chaque occurrence retrouvée, on calcule $\Delta t = t_{\text{database}} - t_{\text{query}}$. Si plusieurs fingerprints indépendants d'un même morceau donnent presque le même Δt , ils sont temporellement cohérents.

9.3. Histogramme

Pour un morceau r , on peut compter les votes dans des intervalles de décalage. Un maximum δ^* est recherché. Une vraie correspondance produit une concentration de votes ; un faux candidat produit plus souvent des offsets dispersés.

9.4. Score simplifié

$\text{Score}(r) = V_r(\delta^*) / \max(1, |Q|)$, où V est le nombre de votes alignés et $|Q|$ le nombre de fingerprints de la requête. Dans un système réel, le score peut intégrer davantage de critères.

10. Pseudo-code complet

Algorithme 1 — construction de la bibliothèque

```
BUILD_INDEX(library):
    index = empty_map()

    for reference in library:
        x = decode(reference)
        x = preprocess(x)
        X = STFT(x, fs, N, H, window)
        S = power_spectrogram(X)
        P = peak_picking(S, threshold, neighborhood)

        for p in P:
            for q in select_targets(P, p):
                dt = q.time - p.time
                key = HASH(Q(p.freq),
                           Q(q.freq),
                           Q(dt))
                index[key].append(
                    (reference.id, p.time))

    return index
```

Algorithme 2 — identification

```
RECOGNIZE(query, index):
    x = preprocess(query)
    X = STFT(x, fs, N, H, window)
    S = power_spectrogram(X)
    P = peak_picking(S, threshold, neighborhood)

    votes = map()

    for p in P:
        for q in select_targets(P, p):
            dtq = q.time - p.time
            key = HASH(Q(p.freq), Q(q.freq), Q(dtq))

            for (track, td) in index[key]:
                delta = td - p.time
                votes[track].append(delta)

    for track in votes:
        delta_star = dominant_offset(votes[track])
        score[track] = aligned_votes(
            votes[track], delta_star)

    return argmax(score)
```

Cette décomposition sépare clairement le prétraitement de la base et le traitement temps réel de la requête.

11. Analyse de complexité

11.1. STFT

Pour M fenêtres de N échantillons, le coût est approximativement $O(MN \log N)$ avec une FFT rapide.

11.2. Peak picking

Une implémentation naïve inspecte une grande partie de la matrice temps-fréquence. Le coût dépend du nombre de cellules et de la taille du voisinage.

11.3. Fingerprints

Si P pics génèrent en moyenne W relations, le nombre de fingerprints est environ $O(PW)$. Augmenter W améliore la redondance mais augmente le stockage et les recherches.

11.4. Recherche

Une recherche de clé dans une table de hachage est en moyenne proche de $O(1)$, mais la reconnaissance complète n'est pas $O(1)$. Il faut parcourir les occurrences associées, calculer les offsets et effectuer le vote. La complexité réelle dépend donc des collisions et de la distribution des clés.

11.5. Mémoire

Si H est le nombre total de fingerprints stockés, la mémoire est de l'ordre de $O(H)$, avec une constante liée à la taille de la clé, de l'identifiant et du timestamp.

Phase	Complexité conceptuelle
STFT	$O(MN \log N)$
Détection de pics	$\approx O(MK)$ naïf
Création des relations	$\approx O(PW)$
Construction index	$O(H)$
Lookup d'une clé	$O(1)$ moyen, hors occurrences
Vote final	Dépend du nombre de candidats

12. Robustesse

12.1. Bruit

Le bruit ajoute de l'énergie mais ne détruit pas nécessairement les maxima dominants. La sélection parcimonieuse permet de conserver des structures suffisamment fortes.

12.2. Variation de volume

Une multiplication globale du signal modifie les amplitudes sans forcément déplacer les positions des pics. Une représentation centrée sur la géométrie des pics peut donc être plus stable qu'une comparaison d'amplitudes brutes.

12.3. Compression

La compression avec pertes peut supprimer des détails, mais suffisamment de points discriminants peuvent rester pour produire des votes cohérents.

12.4. Réverbération

Une capture dans une pièce mélange son direct, réflexions et bruit. Apple indique que ShazamKit peut trouver des correspondances à partir d'un son capturé bruité, par exemple une musique jouée en arrière-plan dans un restaurant.

12.5. Robustesse $\neq$ invulnérabilité

Une bonne empreinte n'est pas immune à toutes les transformations. Des changements importants de tempo, de hauteur, de mixage ou une perturbation extrême peuvent réduire fortement le nombre de correspondances.

13. Expérience numérique pédagogique

Paramètres de démonstration : $f_s=8$ kHz, $N=1024$, $H=512$, résolution fréquentielle $\approx 7,8$ Hz. Ces valeurs servent uniquement à visualiser la méthode.

Supposons trois fingerprints d'une requête. Pour Track A, la base renvoie :

Requête t	Base t	Δt
0,10 s	12,40 s	12,30 s
0,23 s	12,53 s	12,30 s
0,38 s	12,68 s	12,30 s

Les trois offsets sont identiques : 12,30 s. Cette concentration est un signal très fort d'alignement. Pour un mauvais candidat, les offsets seraient par exemple 7,1 s, 19,8 s et 3,4 s : aucune concentration ne se formerait.

Un classement illustratif peut ensuite afficher Track A 94 %, Track C 21 %, Track B 14 %, Track D 7 %. Ces valeurs sont pédagogiques et ne représentent pas une confiance officielle de Shazam.

14. Applications industrielles

Shazam / Apple ShazamKit

ShazamKit permet de faire correspondre un son capturé avec le catalogue Shazam ou un catalogue personnalisé. Apple documente aussi des catalogues personnalisés permettant de synchroniser du contenu avec des timecodes.

ACRCloud

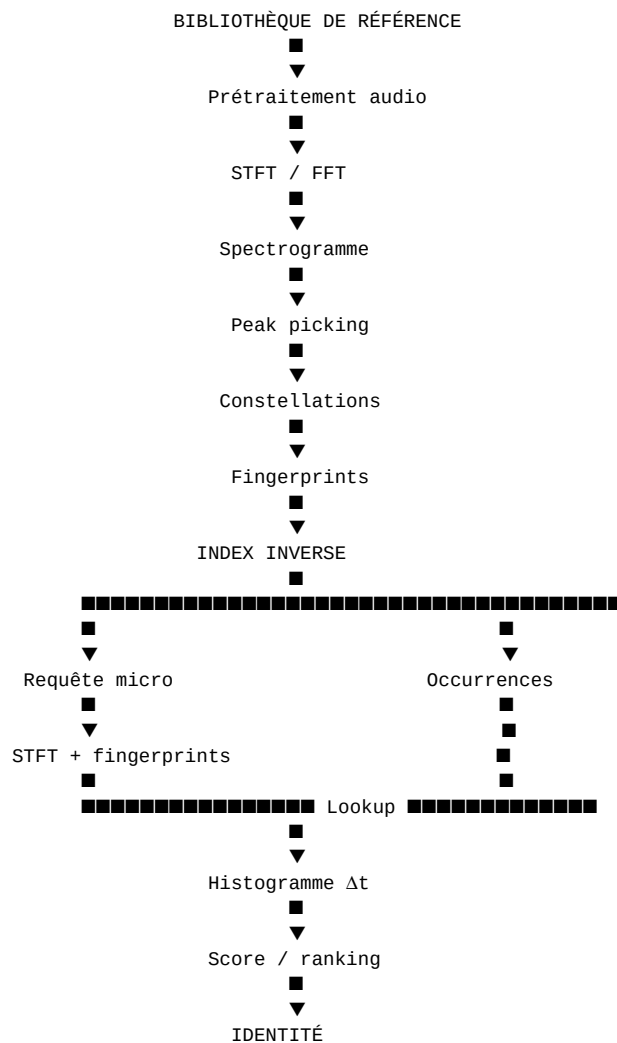
ACRCloud propose de la reconnaissance musicale, du monitoring de diffusion, des fonctions liées au copyright, de la synchronisation second écran, de la reconnaissance de fredonnement et de la reconnaissance hors ligne. Son site annonce une base de plus de 150 millions de titres pour la reconnaissance musicale.

Autres domaines

Domaine	Application
Musique	Identification de chansons
Radio/TV	Monitoring des diffusions
Droits	Détection de contenus protégés
Second écran	Synchronisation avec une émission
Éducation	Déclenchement de contenus selon un timecode
Automobile	Reconnaissance de contenus dans l'habitacle
Événementiel	Identification locale de contenus préchargés

Précaution : ces entreprises utilisent des technologies d'ACR/fingerprinting apparentées ; cela ne signifie pas qu'elles utilisent toutes exactement le même algorithme ou les mêmes paramètres que Shazam.

15. Architecture complète d'un système



La partie supérieure est généralement préparée hors ligne. La partie requête est exécutée lorsqu'un utilisateur demande une identification. Cette séparation est fondamentale pour passer à l'échelle.

Pour des catalogues personnalisés, Apple documente la possibilité de stocker des signatures de référence et leurs métadonnées, puis de rechercher localement des correspondances selon la plateforme et le scénario.

16. Limites, sécurité et confidentialité

16.1. Limites algorithmiques

Une base ne peut identifier que ce qu'elle contient. Une empreinte trop dense augmente la mémoire ; une empreinte trop rare diminue la robustesse. Les paramètres doivent donc être optimisés expérimentalement.

16.2. Limites de connaissance

Il faut distinguer la littérature scientifique, les principes publics de ShazamKit et l'implémentation propriétaire actuelle. Un exposé sérieux doit dire explicitement quand une formule est une reconstruction pédagogique.

16.3. Confidentialité

Une architecture peut fonctionner localement ou sur serveur. Dans les deux cas, il faut contrôler la conservation des requêtes, les accès au catalogue, le chiffrement et les métadonnées. Apple décrit les signatures ShazamKit comme non inversibles vers l'enregistrement original.

16.4. Sécurité de l'index

L'index doit être protégé contre les accès non autorisés et contre les manipulations de contenu. Les systèmes de production peuvent aussi limiter les requêtes, journaliser les erreurs et surveiller les comportements anormaux.

17. Protocole expérimental pour une étude scientifique

Pour transformer la démonstration en véritable expérimentation, il faut créer un jeu de données avec plusieurs genres et plusieurs conditions de capture.

Facteur	Exemples
Bruit	SNR élevé, moyen, faible
Volume	-20, -10, 0, +10 dB
Compression	MP3/AAC à plusieurs débits
Environnement	Pièce, extérieur, haut-parleur
Durée requête	2 s, 5 s, 10 s, 15 s
Type de contenu	Genres et productions variés

Les mesures recommandées sont : accuracy, précision, rappel, taux de faux positifs, latence, taille de l'index et débit de requêtes. Les résultats doivent être séparés en données de réglage et données de test.

Une étude plus avancée peut comparer plusieurs stratégies de peak picking, plusieurs tailles de fenêtre, différentes densités de fingerprints et différentes méthodes de vote temporel.

18. Discussion

La contribution fondamentale de l'acoustic fingerprinting est le changement de représentation. Le système ne cherche pas à comprendre la chanson comme un humain ; il cherche une structure numérique rare, répétable et indexable.

Le point clé est l'alignement temporel. Une collision isolée de hash n'est pas une preuve. Des dizaines de relations différentes qui convergent vers le même Δt constituent une preuve beaucoup plus forte.

Cette architecture associe donc quatre disciplines : traitement du signal pour extraire les caractéristiques ; mathématiques pour modéliser temps et fréquence ; structures de données pour l'index ; statistiques/vote pour décider entre les candidats.

La réussite industrielle dépend ensuite de l'ingénierie : bases de données massives, distribution, cache, parallélisation, gestion des catalogues, contrôle de la latence et qualité des métadonnées.

19. Conclusion

L'algorithme étudié peut être résumé par : **son** → **échantillons** → **STFT** → **spectrogramme** → **pics** → **constellation** → **fingerprints** → **index inversé** → **offsets** → **score** → **identification**.

La STFT fournit le plan temps-fréquence ; le peak picking produit une représentation parcimonieuse ; la constellation encode des relations ; le hash rend ces relations recherchables ; l'index inversé évite une comparaison exhaustive ; l'histogramme des offsets vérifie la cohérence temporelle.

Apple confirme publiquement le principe général d'une signature acoustique compacte fondée sur la distribution temps-fréquence de l'énergie et la recherche de signatures de requête dans un catalogue. Les travaux industriels et les solutions ACR montrent que ce paradigme est devenu une brique réutilisable bien au-delà de la reconnaissance musicale.

Pour un projet pédagogique, l'implémentation de cette chaîne constitue une excellente démonstration de la façon dont une idée mathématique — la représentation temps-fréquence — peut devenir un système industriel de recherche à grande échelle.

20. Références bibliographiques et documentaires

- [1] Avery Wang, “An Industrial Strength Audio Search Algorithm”, ISMIR 2003. Travail de référence sur la recherche audio industrielle et les fingerprints temps-fréquence.
- [2] Apple Developer, “ShazamKit”. Documentation officielle sur les signatures acoustiques, catalogues et matching audio.
- [3] Apple Developer, “SHCustomCatalog”. Documentation sur les catalogues audio personnalisés.
- [4] Apple Developer, “Building a Custom Catalog and Matching Audio”. Exemple de synchronisation de contenu pédagogique.
- [5] Apple Developer, WWDC22, “Create custom catalogs at scale with ShazamKit”. Considérations sur les catalogues, signatures et contenus synchronisés.
- [6] ACRCLOUD, “Music Recognition”. Documentation et présentation de la reconnaissance musicale et des usages ACR.
- [7] ACRCLOUD Documentation, “Identification API”. Documentation de l'identification par audio ou fingerprint.
- [8] ACRCLOUD, “Offline Recognition”. Reconnaissance de contenus personnalisés sans connexion réseau.

21. Glossaire

Terme	Définition
ACR	Automatic Content Recognition : reconnaissance automatique de contenu.
API	Application Programming Interface : interface permettant à un logiciel d'utiliser un service.
DFT	Discrete Fourier Transform : transformée de Fourier discrète.
FFT	Fast Fourier Transform : algorithmes rapides de calcul de DFT.
Fingerprint	Empreinte numérique compacte extraite du contenu audio.
f	Fréquence, exprimée en Hz.
f _s	Fréquence d'échantillonnage.
Hash	Clé compacte permettant l'indexation d'une caractéristique.
Hz / kHz	Hertz / kilohertz, unités de fréquence.
Offset	Décalage temporel entre requête et référence.
Peak picking	Détection de maxima locaux dans le spectrogramme.
STFT	Short-Time Fourier Transform : transformée de Fourier à court terme.
Δt	Différence temporelle utilisée pour aligner les correspondances.

Note finale : ce papier est conçu pour servir à la fois de document scientifique, de support de présentation et de base pour une implémentation pédagogique en Python. Les paramètres numériques sont illustratifs sauf indication contraire.